

5 ПРАКТИЧЕСКАЯ РАБОТА (ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ)

5.1 Цель работы

Целью практической работы является формирование навыков логического проектирования базы данных на основе ранее сформированных требований и моделей предметной области, включая выделение сущностей и атрибутов, определение идентификаторов, установление связей и кардинальностей, разрешение связей типа «многие ко многим» и выполнение нормализации структуры данных.

5.2 Задачи работы

В процессе выполнения практической работы необходимо:

- проанализировать результаты практических работ № 1–4;
- определить состав данных, необходимый для реализации требований проектируемой системы;
- выделить основные сущности предметной области;
- определить назначение каждой сущности;
- определить состав атрибутов сущностей;
- определить идентификаторы сущностей;
- установить связи между сущностями;
- определить кардинальность и обязательность связей;

- разрешить связи типа M:N посредством введения ассоциативных сущностей;
- выявить и устранить избыточность структуры данных;
- выполнить нормализацию отношений;
- построить логическую модель базы данных;
- проверить согласованность модели с требованиями, бизнес-процессами и UML-моделями;
- подготовить логическую модель к последующей физической реализации средствами выбранной СУБД.

5.3 Теоретические сведения

Проектирование базы данных представляет собой последовательный процесс формирования структуры данных, обеспечивающей хранение информации о предметной области и выполнение предусмотренных системой операций.

В рамках проектирования целесообразно различать:

- концептуальное представление предметной области;
- логическую модель данных;
- физическую модель базы данных.

Концептуальное представление определяет основные объекты предметной области и отношения между ними без привязки к особенностям конкретной СУБД.

Логическая модель определяет структуру данных в терминах сущностей, атрибутов, идентификаторов, отношений и ограничений.

Физическая модель определяет способ непосредственной реализации логической структуры средствами конкретной СУБД: таблицы, типы данных, первичные и внешние ключи, индексы, ограничения и другие физические объекты.

В настоящей практической работе выполняется именно логическое проектирование.

Поэтому на данном этапе основное внимание должно уделяться корректному представлению предметной области, а не особенностям синтаксиса конкретной СУБД.

5.3.1 Сущности

Сущность представляет собой выделяемый объект, понятие, событие или явление предметной области, сведения о котором необходимо хранить в информационной системе.

Примерами сущностей могут являться:

- клиент;
- заказ;
- товар;
- сотрудник;
- подразделение;
- заявка;
- платёж;
- документ;
- оборудование;
- бронирование.

Сущность должна обладать самостоятельным смысловым значением в рамках предметной области.

Не следует создавать отдельную сущность только потому, что соответствующее понятие может быть представлено отдельной таблицей.

Например, значения:

- фамилия;
- имя;
- номер телефона;
- дата рождения

обычно являются атрибутами сущности Пользователь, а не самостоятельными сущностями.

При выделении сущности рекомендуется ответить на следующие вопросы:

- необходимо ли хранить множество экземпляров данного объекта;
- имеет ли объект собственные характеристики;
- участвует ли объект в отношениях с другими объектами;
- необходима ли информация об объекте для реализации требований системы;
- должен ли объект существовать независимо от отдельных операций пользовательского интерфейса.

5.3.2 Атрибуты

Атрибут представляет собой характеристику сущности.

Например, сущность Заявка может содержать атрибуты:

- номер;
- дата создания;

- описание;
- состояние;
- плановый срок выполнения.

Каждый атрибут должен характеризовать непосредственно ту сущность, в составе которой он размещён.

Не рекомендуется хранить в одном атрибуте несколько независимых значений.

Например, значение:

«Иванов Иван Иванович, +7-999-111-22-33»

не должно рассматриваться как единый атрибут, если фамилия, имя и телефон используются системой как самостоятельные характеристики.

При формировании состава атрибутов необходимо избегать:

- повторяющихся групп;
- хранения нескольких однотипных значений в одном поле;
- необоснованного дублирования информации;
- хранения производных значений, которые могут быть однозначно рассчитаны на основании других данных, если отсутствует необходимость их самостоятельного хранения.

5.3.3 Идентификаторы сущностей

Каждый экземпляр сущности должен иметь возможность однозначной идентификации.

Для этого определяется ключевой атрибут либо совокупность атрибутов.

Различают:

- потенциальный ключ;

- первичный ключ;
- составной ключ;
- естественный ключ;
- искусственный (суррогатный) ключ.

Потенциальный ключ представляет собой минимальный набор атрибутов, позволяющий однозначно идентифицировать экземпляр сущности.

Из множества потенциальных ключей один может быть выбран в качестве первичного.

Естественный ключ образуется на основании характеристик, существующих в предметной области.

Примерами могут быть:

- номер документа;
- артикул;
- государственный регистрационный номер.

Суррогатный ключ представляет собой специально введённый технический идентификатор, например `client_id`, `order_id`.

Использование суррогатного ключа не освобождает от необходимости учитывать естественную уникальность данных.

Например, если в предметной области номер договора не может повторяться, введение `contract_id` не отменяет требования уникальности номера договора.

5.3.4 Связи между сущностями

Связь отражает смысловое отношение между экземплярами сущностей.

Для каждой связи необходимо определить:

- участвующие сущности;
- смысл отношения;
- кардинальность;
- обязательность участия;
- при необходимости — собственные атрибуты связи.

Наиболее распространёнными являются следующие типы связей.

Связь 1:1

Одному экземпляру первой сущности соответствует не более одного экземпляра второй сущности и наоборот.

Подобная связь должна иметь содержательное обоснование.

Не следует искусственно разделять одну сущность на несколько сущностей только для создания отношения 1:1.

Связь 1:M

Одному экземпляру первой сущности может соответствовать несколько экземпляров второй сущности.

Например:

один клиент может оформить несколько заказов, при этом каждый заказ относится к одному клиенту.

Такой тип отношения является одним из наиболее распространённых в реляционных моделях.

Связь M:N

Нескольким экземплярам первой сущности могут соответствовать несколько экземпляров второй сущности.

Например:

один заказ может содержать несколько товаров, а один товар может входить в несколько заказов.

В реляционной модели непосредственная реализация связи M:N не используется.

Такая связь должна быть разрешена посредством введения промежуточной, или ассоциативной, сущности.

Например:

Заказ – Позиция заказа – Товар

Ассоциативная сущность может содержать собственные атрибуты, характеризующие конкретный факт связи.

Для позиции заказа это могут быть:

- количество;
- цена на момент оформления;
- размер скидки.

5.3.5 Кардинальность и обязательность связей

Кардинальность определяет количество экземпляров одной сущности, которые могут быть связаны с экземпляром другой сущности.

Типичными вариантами являются:

- 1;
- 0..1;
- 1..*;
- 0..*.

Например: каждый заказ должен принадлежать ровно одному клиенту;

со стороны клиента: клиент может не иметь заказов либо иметь несколько заказов.

Следовательно, отношения с разных сторон имеют различную обязательность.

Кардинальность должна определяться правилами предметной области.

Не допускается выбор кардинальностей исключительно исходя из удобства последующей реализации.

5.3.6 Нормализация данных

Нормализация представляет собой процесс преобразования структуры отношений, направленный на уменьшение избыточности данных и предотвращение аномалий их обработки.

Избыточная структура может приводить к следующим проблемам:

- аномалия вставки;
- аномалия обновления;
- аномалия удаления.

Первая нормальная форма

Отношение находится в первой нормальной форме, если:

- каждый атрибут содержит одно значение;
- отсутствуют повторяющиеся группы;
- значения атрибутов являются атомарными в контексте решаемой задачи.

Например, хранение списка телефонов:

+79990000001; +79990000002; +79990000003

в одном атрибуте препятствует нормальной работе с отдельными номерами.

Если система должна обрабатывать телефонные номера независимо, они должны быть представлены отдельными значениями.

Вторая нормальная форма

Отношение находится во второй нормальной форме, если:

- оно находится в первой нормальной форме;
- каждый неключевой атрибут полностью зависит от всего первичного ключа.

Требование второй нормальной формы имеет особое значение для отношений с составным ключом.

Например, если отношение содержит ключ:

(order_id, product_id)

то атрибут, характеризующий исключительно товар и не зависящий от конкретного заказа, не должен храниться в этом отношении.

Третья нормальная форма

Отношение находится в третьей нормальной форме, если:

- оно находится во второй нормальной форме;
- отсутствуют транзитивные зависимости неключевых атрибутов от первичного ключа.

Иными словами, неключевой атрибут не должен определяться другим неключевым атрибутом, если такая зависимость представляет самостоятельное понятие предметной области.

В рамках настоящей практической работы логическая модель должна быть приведена не ниже третьей нормальной формы, если отступление от нормализации не имеет содержательного и технически обоснованного назначения.

5.3.7 Денормализация

В реальных информационных системах в отдельных случаях может применяться денормализация — осознанное введение определённой избыточности данных для решения конкретной задачи.

Однако денормализация не является основанием для сохранения ошибок проектирования.

Если обучающийся использует структуру, не соответствующую третьей нормальной форме, необходимо обосновать:

- какое нарушение нормализации присутствует;
- с какой целью оно сохранено;
- какие преимущества оно обеспечивает;
- какие риски несогласованности данных возникают;
- каким образом обеспечивается корректность данных.

Формулировка:

«так было удобнее»

не является достаточным обоснованием денормализации.

5.3.8 Отличие логической модели от UML-диаграммы классов

Результаты практической работы № 4 являются одним из источников информации для проектирования модели данных, однако UML-диаграмма классов и логическая модель базы данных решают различные задачи.

UML-диаграмма классов описывает статическую структуру программной или предметной модели.

Логическая модель базы данных определяет организацию информации, подлежащей хранению в СУБД.

Поэтому между UML-классами и сущностями базы данных не обязательно существует отношение 1:1.

В процессе перехода к логической модели могут:

- появиться ассоциативные сущности;
- объединиться отдельные концептуальные элементы;
- выделиться самостоятельные справочные сущности;
- измениться структура атрибутов;
- появиться идентификаторы;
- уточниться кардинальности;
- исключиться элементы, не предназначенные для постоянного хранения.

Подобные изменения должны быть результатом анализа, а не механического преобразования одной диаграммы в другую.

5.4 Задание

На основании требований, бизнес-процессов и UML-моделей необходимо разработать логическую модель базы данных проектируемой системы.

Модель должна охватывать данные, необходимые для реализации основных функций системы.

5.4.1 Выделение сущностей

Необходимо:

- проанализировать основные группы данных;
- проанализировать UML-классы;
- проанализировать информационные объекты бизнес-процесса;
- определить сущности, сведения о которых должны храниться в базе данных;
- дать каждой сущности однозначное наименование;
- кратко определить назначение каждой сущности.

5.4.2 Определение атрибутов

Для каждой сущности необходимо определить:

- идентификатор;
- основные обязательные характеристики;
- дополнительные характеристики, необходимые для реализации требований.

Не следует включать атрибуты, не используемые ни одним требованием, процессом или предполагаемым сценарием системы без соответствующего обоснования.

5.4.3 Определение ключей

Для каждой сущности необходимо определить:

- первичный идентификатор;
- при наличии — альтернативные уникальные идентификаторы.

Выбор ключа должен быть обоснован семантикой данных.

5.4.4 Определение отношений

Для связанных сущностей необходимо:

- определить смысл отношения;
- определить кардинальность;
- определить обязательность;
- определить необходимость хранения дополнительных характеристик отношения.

Все связи M:N должны быть разрешены посредством ассоциативных сущностей.

5.4.5 Нормализация

Необходимо проверить полученные отношения на соответствие:

- первой нормальной форме;
- второй нормальной форме;
- третьей нормальной форме.

При выявлении нарушения модель должна быть преобразована.

Если намеренно сохраняется денормализованная структура, необходимо привести письменное обоснование.

5.4.6 Построение логической модели

По результатам анализа необходимо построить ER-диаграмму либо эквивалентную логическую модель данных.

Допускается использование нотаций:

- Crow's Foot;
- IDEF1X;
- иной стандартной нотации моделирования данных по согласованию с преподавателем.

В рамках одной модели должна использоваться единая система обозначений.

5.5 Минимальные требования к результату

Для зачёта практической работы логическая модель должна содержать:

- не менее 6 содержательных сущностей;
- первичный идентификатор для каждой сущности;
- содержательный набор атрибутов для каждой основной сущности;
- не менее 5 связей между сущностями;
- указание кардинальности связей;
- указание обязательности участия там, где это необходимо;
- корректное разрешение всех связей M:N;
- структуру, приведённую не ниже третьей нормальной формы, либо обоснование отступления от неё;
- данные, необходимые для реализации основных требований системы.

Количество сущностей и связей является минимальным количественным условием и не заменяет требования содержательной полноты.

Не учитываются при определении минимального количества сущности, созданные исключительно для искусственного усложнения модели.

Например, необоснованное разделение:

Пользователь → Имя пользователя → Контакт пользователя → Адрес пользователя

на несколько независимых сущностей не рассматривается как повышение сложности модели.

5.6 Требования к качеству логической модели

Разработанная модель должна обладать следующими свойствами.

Полнота

В модели должны присутствовать данные, необходимые для реализации основных требований проектируемой системы.

Непротиворечивость

Один и тот же факт предметной области не должен одновременно храниться в нескольких независимых местах без обоснованной необходимости.

Однозначность

Назначение сущностей, атрибутов и связей должно быть понятно из их наименований и структуры модели.

Минимизация избыточности

Следует исключить необоснованное дублирование информации.

Целостность

Структура должна позволять определить правила сохранения корректных связей между информационными объектами.

Расширяемость

Модель не должна быть построена таким образом, чтобы очевидное развитие заявленной функциональности требовало полной перестройки структуры данных.

5.7 Проверка согласованности с предыдущими практическими работами

После разработки логической модели необходимо выполнить проверку её согласованности с материалами ПР №1–4.

Следует установить:

- все основные группы данных ПР №1 представлены в модели;
- требования ПР №2 имеют необходимое информационное обеспечение;
- информационные объекты ПР №3 представлены в модели либо обоснованно являются временными;
- основные классы предметной области из ПР №4 сопоставлены с сущностями;
- отношения между сущностями не противоречат UML-модели;
- роли и пользовательские сценарии имеют необходимые данные для выполнения операций;
- бизнес-правила могут быть реализованы на основании представленной структуры.

Если логическое проектирование выявило ошибку ранее разработанных моделей, необходимо скорректировать соответствующие материалы.

5.8 Проверка модели на возможность реализации требований

Для каждого основного функционального требования необходимо определить, какие сущности участвуют в его реализации.

Например:

Требование	Используемые сущности
FR-01. Зарегистрировать заявку	Пользователь, Заявка, Категория заявки
FR-04. Назначить исполнителя	Заявка, Сотрудник
FR-08. Получить историю изменений	Заявка, История состояния

Если для выполнения требования отсутствуют необходимые данные, логическая модель является неполной.

Если значительная часть сущностей не связана ни с одним требованием, необходимо проверить обоснованность их наличия.

5.9 Порядок выполнения работы

- Проанализировать замечания по ПР №4.
- Изучить требования к данным из технического задания.
- Выписать основные информационные объекты.
- Сопоставить их с UML-классами.
- Выделить первоначальный набор сущностей.
- Определить назначение каждой сущности.
- Определить атрибуты.
- Определить потенциальные и первичные ключи.
- Определить отношения между сущностями.
- Определить кардинальности.

- Определить обязательность участия.
- Разрешить связи M:N.
- Проверить первую нормальную форму.
- Проверить вторую нормальную форму.
- Проверить третью нормальную форму.
- Устранить выявленную избыточность.
- Построить логическую модель.
- Сопоставить модель с требованиями.
- Сопоставить модель с UML и моделью процесса.
- Исправить выявленные противоречия.
- Подготовить итоговую диаграмму.
- Подготовить отчёт.
- Представить модель преподавателю для прохождения контрольной точки № 2.

5.10 Содержание отчёта

Отчёт по практической работе должен содержать:

- титульный лист;
- номер и название практической работы;
- цель работы;
- краткое описание исходных данных для проектирования;
- перечень выделенных сущностей и их назначение;
- описание ключевых атрибутов;
- сведения о выбранных идентификаторах;

- описание основных связей;
- обоснование кардинальностей;
- описание разрешения связей M:N;
- результаты проверки нормальных форм;
- логическую модель базы данных;
- сопоставление основных требований и сущностей;
- сведения об изменениях предыдущих моделей, если они были внесены;
- вывод по результатам работы.

5.11 Результаты, предоставляемые преподавателю

Для защиты практической работы обучающийся предоставляет:

- отчёт;
- логическую модель в редактируемом формате;
- экспорт логической модели в графический формат или PDF;
- при необходимости — актуализированные UML-модели и техническое задание.

Рекомендуемое размещение итоговой модели:

/diagrams/er_model.png

или в ином согласованном формате.

Обучающийся должен быть готов:

- объяснить назначение любой сущности;
- обосновать размещение любого существенного атрибута;
- объяснить выбор первичного идентификатора;

- обосновать кардинальность любой связи;
- показать разрешение M:N;
- объяснить, почему структура соответствует третьей нормальной форме;
- указать требования, для реализации которых используется выбранная сущность;
- показать соответствие модели предыдущим проектным материалам.

5.12 Использование результатов в курсовой работе

Логическая модель является одним из центральных проектных результатов курсовой работы и непосредственно определяет последующую структуру базы данных.

Для зачёта настоящей практической работы достаточно модели, содержащей не менее шести содержательных сущностей и соответствующей установленным требованиям.

Для курсовой работы логическая модель должна охватывать весь значимый состав хранимых данных итоговой системы.

Ориентировочно для курсовой работы рекомендуется наличие не менее 8 содержательных сущностей либо модели сопоставимой структурной сложности.

Указанное количество не является самостоятельной целью проектирования. Предметная область может объективно требовать и большего количества сущностей.

Не допускается искусственное увеличение количества таблиц или сущностей исключительно для достижения количественного значения.

В курсовой работе необходимо:

- обосновать выделение основных сущностей;
- определить идентификаторы;
- обосновать наиболее существенные отношения;
- показать разрешение связей M:N;
- привести модель к требуемым нормальным формам;
- при наличии денормализации привести её техническое обоснование;
- обеспечить соответствие модели физически реализованной базе данных.

Следует учитывать, что начиная со следующей практической работы логическая модель становится основой физической реализации.

Следовательно, позднее изменение модели может потребовать изменения:

- SQL-скриптов создания базы данных;
- скриптов наполнения;
- запросов;
- функций;
- процедур;
- триггеров;
- ролей и прав;
- программного интерфейса;
- тест-кейсов.

Поэтому выполнение настоящей работы исключительно на минимальном уровне с расчётом на последующее расширение может привести к значительному увеличению объёма переработки курсового проекта.

Если предметная область уже рассматривается как основа курсовой работы, рекомендуется на ПР №5 формировать модель, максимально близкую к предполагаемой итоговой структуре.

5.13 Контрольная точка № 2. Фиксация базовой модели данных

Защита практической работы № 5 является второй контрольной точкой сквозного проекта.

После успешной защиты:

- утверждается базовый состав сущностей;
- утверждаются основные отношения между сущностями;
- фиксируется логическая структура данных;
- модель становится основанием для физического проектирования базы данных.

После прохождения контрольной точки допускаются уточнения структуры данных.

Однако существенное изменение модели, включая:

- удаление основных сущностей;
- добавление значительного количества новых сущностей;
- изменение ключевых отношений;
- изменение логики хранения основных информационных объектов,

требует актуализации всех связанных проектных материалов.

В частности, может потребоваться корректировка:

- технического задания;
- модели бизнес-процесса;
- UML-диаграмм;
- перечня требований.

Начиная с ПР №6 изменение логической модели дополнительно потребует актуализации реализованной структуры базы данных.

5.14 Контрольные вопросы

- В чём заключается различие между концептуальной, логической и физической моделью данных?
- Что представляет собой сущность?
- По каким признакам атрибут отличается от самостоятельной сущности?
- Для чего используется первичный ключ?
- Чем естественный ключ отличается от суррогатного?
- Почему введение суррогатного ключа не отменяет естественных ограничений уникальности?
- Какие типы кардинальностей используются при моделировании данных?
- Каким образом разрешается связь M:N в реляционной модели?
- В каких случаях ассоциативная сущность имеет собственные атрибуты?
- Что представляет собой первая нормальная форма?
- Что представляет собой вторая нормальная форма?
- Что представляет собой третья нормальная форма?
- Какие аномалии могут возникать вследствие избыточности данных?
- Что представляет собой денормализация?
- В каких случаях денормализация может быть оправдана?
- Почему UML-диаграмма классов не должна автоматически преобразовываться в набор таблиц?
- Каким образом требования системы используются при проверке полноты модели данных?

- Какие последствия может вызвать существенное изменение логической модели после начала физической реализации базы данных?

5.15 Критерии приёмки работы

Практическая работа может быть зачтена при одновременном выполнении следующих условий:

- представлена логическая модель базы данных;
- выделено не менее 6 содержательных сущностей;
- для всех сущностей определены идентификаторы;
- основные сущности содержат необходимые атрибуты;
- представлено не менее 5 содержательных связей;
- определены кардинальности;
- связи M:N корректно разрешены;
- модель соответствует не ниже третьей нормальной форме либо представлены обоснованные исключения;
- структура данных обеспечивает выполнение основных требований системы;
- модель согласована с ПР №1–4;
- отсутствует искусственное увеличение количества сущностей;
- отсутствует необоснованное дублирование данных;
- обучающийся способен объяснить структуру модели;
- обучающийся способен обосновать ключи, связи и кардинальности;
- обучающийся способен объяснить результаты нормализации.

После успешной защиты практической работы считается пройденной контрольная точка № 2.

5.16 Срок выполнения и сдачи

Срок выполнения и защиты практической работы определяется календарным графиком практических занятий.

Практическая работа № 5 должна быть защищена до начала физического проектирования и создания базы данных.

Обучающийся, не имеющий согласованной логической модели данных, не должен переходить к окончательной реализации физической структуры базы данных, поскольку последующие SQL-скрипты должны разрабатываться на основании утверждённой модели.